

本地记忆方案对比：CC vs OpenClaw vs 云记忆

从纯文本到语义检索，再到云端服务 — 三个演进阶段

Claude Code

纯 Markdown 文件

- MEMORY.md + 独立记忆文件
- 检索：grep / 文件名匹配
- 200 行截断，需人工修剪
- 可 git 同步，但无语义理解
- 单人单项目，跨项目靠手动

类比：记事本

OpenClaw

Markdown + SQLite 向量

- Markdown 存储 + sqlite-vec 向量索引
- 混合检索：70% 语义 + 30%BM25
- 本地 .sqlite 文件，不可网络共享
- SQLite 单进程，无并发写入
- 索引是二进制，无法 git 同步

类比：个人笔记软件

Cloud Memory

云端记忆服务 (DBay)

- 云端向量 + 全文 + 结构化检索
- API 接入，任何 Agent 可调用
- 多用户、多项目、多 Agent 共享
- 租户隔离 + 权限控制 + 审计日志
- 记忆属于用户，不属于 Agent

类比：企业知识平台

CC 的 Markdown 能 git 同步但无语义 · OpenClaw 有语义但 SQLite 锁死单机 · 两者都困在本地范式

痛点："养 Agent" — 每个 Agent 的记忆从零开始

OpenClaw 需要 1-2 周才真正好用 · 多 Agent 记忆完全割裂 · 用户反复付出训练成本

"养"一个 Agent 的时间线

Day 1 > Day 3 > Day 7 > Day 14
不了解用户 > 记住偏好 > 理解模式 > 终于好用

每次纠正 = 用户在付出训练成本，而这些经验无法迁移到其他 Agent

多 Agent 记忆完全割裂

CC 里说 "不要 mock 数据库" — OpenClaw 不知道

OpenClaw 里纠正代码风格 — CC 不知道

两个都要养 = 双倍训练成本，零倍协同收益

一个开发者的 Agent 记忆孤岛

CC 终端编程

OpenClaw 桌面

Cursor IDE

企业内部 Agent

每个 Agent 独立记忆池

换一个 Agent = 记忆清零

N 个 Agent = N 倍养育成本

根本原因：记忆被锁在 Agent 里

记忆绑定在 Agent 内部，而不是属于用户本人

就像通讯录锁在微信里 — 换个聊天软件就丢了所有联系人

记忆不应该属于 Agent，记忆应该属于用户

企业场景的致命缺陷：团队协作、安全合规、非编程 Agent

CC / OpenClaw 的本地记忆方案在企业环境下全面失效

知识无法共享与传承

团队成员间无法共享记忆，重复踩坑
员工离职 = 记忆随笔记本消失，经验归零

安全审计不可能

无法知道 Agent 记住了哪些敏感信息
无法统一擦除（离职 / 合规要求时）
敏感数据散落 N 台开发机，安全部门噩梦

SQLite 架构天花板 (OpenClaw)

单进程数据库，10 人团队无法共享
二进制索引无法 git 同步
不同机器的 embedding 向量不兼容

非编程 Agent — 更大的市场

客服
Agent

HR Agent

销售
Agent

运维
Agent

法务
Agent

财务
Agent

这些 Agent 是云原生 SaaS 服务，根本没有本地文件系统

Markdown / SQLite 方案完全不适用

企业需求 vs 本地记忆能力

需求	本地方案	云端方案
多人协作	不支持	原生支持
权限控制	不支持	租户 + 角色
审计日志	不支持	全链路追踪
数据擦除	逐机清理	一键删除
备份恢复	靠个人	自动备份

解法：云端记忆层 — 记忆属于用户，不属于 Agent

一次养育所有 Agent 共享 · 团队知识资产化 · 跨设备跨平台

核心范式转换

本地范式：记忆 = Agent 的私有状态（换 Agent 就丢失）

云端范式：记忆 = 用户的数字资产（跨 Agent、跨设备、可传承）

对个人开发者

CC 里的纠正，OpenClaw 自动知道

新项目即刻继承所有历史偏好

不用再“养”任何 Agent

架构：Agent-agnostic Memory Layer



对企业团队

新员工继承团队知识，Day 1 即战力

安全部门可审计、可擦除、可管控

非编程 Agent 通过 API 接入同一记忆池

效果对比

每个 Agent 养 2 周

记忆随 Agent 割裂

员工离职知识丢失

安全审计不可控

养一次，处处可用

统一记忆池

团队知识资产沉淀

全链路审计管控

就像从 SQLite 到云数据库 — Agent 记忆必然从本地文件走向云服务

不同 Agent 场景需要不同的记忆策略

5 类 Agent 场景 × 3 层记忆系统 = 场景化 Preset 配置

5 类 Agent 场景



编码助手

Claude Code / Cursor

加载策略
Markdown 全量注入
CLAUDE.md 200 行上限

记忆类型
Decision / Rejection / Pattern

衰减 / 检索
不衰减 (决策长期有效)
无语义检索, 全量加载
Token 预算: <500 tokens

不可编辑 · Git 可追踪



个人助理

OpenClaw / ChatGPT

加载策略
Markdown + SQLite 混合
核心文件全注入 + 历史搜索

记忆类型
Fact / Trait / Episode

衰减 / 检索
指数衰减 $\lambda=0.95$ / 周
Vector+BM25+RRF+MMR
Token 预算: <800 tokens

精准召回 · 节省 72% Token



业务垂直

客服 / 销售 / 法律

加载策略
结构化 DB 按需加载
客户 ID 精确 + 语义补充

记忆类型
Fact / Decision / Episode

衰减 / 检索
不衰减 (审计合规)
ID 精确 + 语义 + RBAC
Token 预算: 按需完整加载

审计轨迹 · Append-only



多 Agent 协作

CrewAI / AutoGen

加载策略
共享状态实时读取
任务 ID 精确查询

记忆类型
Context / Result

衰减 / 检索
任务结束整体清除
任务 ID 精确查询
Token 预算: <200 tokens

跨 Agent 状态共享

统一三层记忆系统 — 同一引擎, 场景化配置



编码助手

策略层
提取 / 衰减 / 合并 / 排序 / 反思

组织层



个人助理

- Markdown 全量
- 不衰减
- 全文匹配

- Decision/Pattern



业务垂直

- 自动提取
- 指数衰减
- Q-value 加权

- Fact/Trait/Episode



多 Agent 协作

- 规则提取
- 不衰减 (合规)
- ID + 语义

- Fact/Decision



多 Agent 协作

- 任务结束提取
- 整体清除
- 任务 ID 精确

- Context/Result

高价值垂直场景举例：面向不同行业 Agent 的记忆策略

不同 Agent 场景需要不同的记忆类型、衰减策略、Trait 反思焦点和隐私等级



客服 / 售后 Agent

Trait 反思焦点

- 情绪敏感度（易怒 / 耐心 / 焦虑）
- 偏好渠道（电话 vs 文字 vs 视频）
- 问题复发模式（同类问题反复出现）

记忆策略

- 投诉历史 + 情绪轨迹时序存储
- 解决方案效果 Q-value 追踪
- 满意度趋势 → 自动升级预警

Q-value 信号清晰：满意度评分直接回传

⚡ 预判客户情绪，主动匹配最优解决路径



销售 / CRM Agent

Trait 反思焦点

- 决策风格（数据驱动 / 关系驱动 / 价格敏感）
- 沟通偏好（简洁直接 vs 详细方案）
- 采购周期节奏和关键决策人

记忆策略

- 客户关系阶段 + 互动时序图谱
- 每次接触的成交贡献度评分
- 竞品提及记录 + 异议模式

最强信号：成交 / 丢单 直接标定记忆价值

⚡ 记住每个客户的决策链路和沟通风格



教育

Trait 反思焦点

- 学习风格（视觉 / 听觉 / 动手）
- 知识薄弱区域
- 注意力持续时间

记忆策略

- 知识点掌握度
- 不同教学策略
- 错题模式 + 易错题

测试成绩 / 答题时长

⚡ 多分支试错



医疗健康 Agent

Trait 反思焦点

- 症状描述习惯（精确 vs 模糊）
- 治疗依从性模式
- 健康焦虑程度和信息需求层次

记忆策略

- 症状时序链 + 用药记录不可变存储



法律 / 合规 Agent

Trait 反思焦点

- 风险偏好（保守 / 激进）
- 关注的合规领域和法规版本
- 案件处理模式和论证风格

记忆策略

- 案例先例图谱 + 判例关联



研究

Trait 反思焦点

- 研究方法论偏好
- 信息源可信度
- 论证推理风格

记忆策略

- 论点演化链

关键技术 1：语义化、结构化的记忆组织层，从碎片文本到知识图谱 + 向量融合

通过注入时增强 + 定期反思整理，让记忆形成语义网络，支撑 Agent 高质量召回

注入时结构化流水线

每条对话进入记忆前，经过 4 步语义增强

1 滑动窗口 + 代词消解

原文: "她讨厌那个框架"

增强: "Alice 讨厌 React 框架"

- 上下文窗口回溯 5 轮
- 消解代词、补全省略实体
- 注入用户属性 (职业等)

参考 HydraDB f_0

2 实体抽取 + 关系图谱

Alice —WORKS_AT—> Meta
Alice —DISLIKES—> React
Meta —USES—> React

- 自动提取实体和关系三元组
- 20+ 关系类型分类标注
- 双向实体查询索引

参考 Supermemory Graph

3 多向量存储

v_content 原文向量

直接语义匹配

v_inferred 增强向量

捕获代词消解后的隐含语义

v_sparse BM25 特征

关键词精确匹配 (罕见词不丢失)

三路融合: $S = \alpha \cdot \text{content} + \beta \cdot \text{inferred} + \gamma \cdot \text{BM25}$

参考 HydraDB 三路检索

4

定期反思与记忆整理

后台异步执行，零运行时开销

A Trait 反思

从 Episode 中发现
用户行为模式
9 步流水线

B 证据聚合

支持 / 反驳证据链
置信度 A/B/C/D 评级
矛盾检测与消解

C 图谱整理

合并冗余实体
层级摘要 L0/L1/L2
智能遗忘低 Q 记忆

Trait 生命周期

trend

candidate

emerging

establish
ed

core

结构化带来的检索能力提升

✗ 传统方式：碎片化存储

"她讨厌那个框架" → 丢失主语和指代
"I moved to the office" → 丢失职业上下文
更新覆盖旧值 → 历史信息永久丢失

~40% 记忆碎片化不可检索

图谱多跳推理

关键技术 2：自演进记忆策略，让系统自己学会如何记忆

不预设记忆策略，而是通过数据飞轮自动发现每个场景的最优记忆方式

核心命题：记忆策略 = 一组可学习的参数（提取什么 / 怎么衰减 / 怎么检索 / 反思什么 / 怎么压缩） → 通过 Agent 使用反馈自

可学习的策略参数

提取策略

提取什么类型？权重如何？

衰减函数

不同类型衰减速率？

检索公式

各信号混合权重？

Trait 焦点

反思什么行为维度？

压缩策略

何时合并 / 归档？

✗ 手工设计策略的问题

- 需要领域专家，成本高
- 用户行为在变，策略会过时
- 无法 A/B 对比策略效果
- 新场景上线周期长（月级）

策略模型
更新参数

自演进飞轮

实时 Phase 1：Q-value 实时反馈

Agent 调用记忆 → 任务成功 / 失败 → Q-value 自动更新

- 成功任务使用的记忆 → Q-value ↑ | 失败任务的记忆 → Q-value ↓
- 更新公式： $Q_{new} = Q_{old} + \alpha \times (reward - Q_{old})$ ，零 GPU，纯数值更新
- MemRL 研究证实：Q-value 加权检索在复杂多步任务提升 56%

Lakebase

Q-value ↓ 轨迹数据

周级 Phase 2：Lakebase 分支 A/B 实验

Copy-on-Write 分支 → 同一用户数据运行不同策略 → 对比效果

- 分支 A：高衰减策略 vs 分支 B：低衰减策略 → 聚合对比胜出上线
- DataLake 聚合 Q-value 轨迹 → 发现场景级模式（如：客服场景情绪记忆 Q-value 显著更高）
- 自动调整：该场景的情绪提取权重 ↑、检索排序权重 ↑

分支实验

AI DataLake

策略对比数据

月/季度 Phase 3：RL 策略模型训练

导出 Q-value 轨迹 + Trait 证据链 → Lance 格式训练数据

- RL 训练策略网络：输入（场景特征 + 记忆数据）→ 输出（最优策略参数向量）